阶段二：vdot.vv 的设计与实现

---

1. 指令编码

编码 `0xFA20A1D7`，按位段拆开：

字段	位	值	含义
`funct6`	`[31:26]`	`111110`	选定的操作码槽
`vm`	`[25]`	`1`	不使用掩码
`vs2`	`[24:20]`	`00001`	第一个源
`vs1`	`[19:15]`	`00010`	第二个源
`funct3`	`[14:12]`	`010`	OPMVV 子类
`vd`	`[11:7]`	`00011`	目的寄存器
`opcode`	`[6:0]`	`1010111`	OP-V 分组

与阶段一的 `vadd.vv`（`0x021101D7`）对比：

	`vadd.vv`	`vdot.vv`
opcode	`1010111`	`1010111`
funct3	`000`（OPIVV）	`010`（OPMVV）
funct6	`000000`	`111110`

同一条 opcode 下，`funct3` 决定子类。点积属于乘加类，所以落在 OPMVV。

---

2. 指令语义

```
a[i] = int8(vs1.byte[i])        # 把字节按 8 位补码解释，最高位是符号位
b[i] = int8(vs2.byte[i])        # 负数保持为负
p[i] = a[i] * b[i]              # 精确的 16 位乘积，范围 [-16256, 16384]
acc  = Σ_{i<8} sext32(p[i])     # 16 位乘积符号扩展到 32 位后累加
vd[31:0]   = acc
vd[127:32] = 0
```

`sext32` 是把 16 位乘积的高位按符号位补齐。硬件上有两次扩展，位置不同：乘法器**之前**，8 位操作数先符号扩展到 16 位（所以实际是 16×16 乘法器）；乘法器**之后**，16 位乘积再 `pad(32)`。两次都是补符号位。这与"先把操作数扩到 32 位再相乘"数值等价，实现上省一截。

三个要点：

1. 固定 8 个元素，`vl` 不参与计算；
2. 整寄存器写回，高 96 位清零；

3. 32 位回绕，不饱和。

与 RVV 0.7.1 同名指令的区别：

维度	RVV 0.7.1 的 vdot.vv	本题
归约形状	逐元素（元素内 EDIV 子元素）	整寄存器 8→1
结果位置	每个 vd[i]	只写 vd[0]，高位清零
掩码	支持 vm	vm=0 非法
重启	支持 vstart	vstart≠0 非法

非法条件四条：vm=0、vsew≠e8、vlmul≠1、vstart≠0。

vdot 这个名字在 RVV 0.7.1（2019）里存在过：语义是把每个元素按 EDIV 再切成子元素做点积，结果写回每个 vd[i]。RVV 1.0（2021 年批准）把它删掉了，没有给出替代指令，整数点积方向交给了未批准的 Zvdot4a8i。

所以现在的 RVV 1.0 手册里查不到 vdot，0.7.1 的手册里才有，而且两版的语义与编码都不同。本作品沿用这个名字，语义按题面的"整寄存器 8→1"重新定义。查表时要注意手册版本，不能把 0.7.1 的编码套到 1.0 上。

---

3. 编码是怎么选出来的

3.1 一开始选错了

最早的候选是 0xC220A1D7。当时的判断方法是在 VecDecoder.scala 里搜 110000 这个 funct6，没搜到，就认为这个槽是空的。

这个方法是错的。VecDecoder.scala 是译码表，里面只有符号名：

```
VWADDU_VV -> OPMVV(T, FuType.vialuF, VialuFixType.vwaddu_vv, F, T, F,
UopSplitType.VEC_VVW)
```

编码本身定义在 rocket-chip 子模块里，是一条 32 位位串，? 为通配：

```
// rocket-chip/src/main/scala/rocket/Instructions.scala:813
def VWADDU_VV = BitPat("b110000??????????010?????1010111")
```

同一串按字段断开。下划线只为看清边界，原始位串里没有：

```
110000_?_?????_?????_010_?????_1010111
```

段	内容	位宽	字段
1	110000	6	funct6
2	?	1	vm
3	?????	5	vs2
4	?????	5	vs1
5	010	3	funct3
6	?????	5	vd
7	1010111	7	opcode

`110000` 就是这条位串的前 6 位。在译码表里搜不到它，因为定义在另一个目录；即便打开定义文件，也要按位段数一遍才认得出。

3.2 选错的代价

`0xC220A1D7` 是 RVV 1.0 已定义的 `vwaddu.vv`：

```
0xC220A1D7  =  funct6 110000  +  funct3 010  +  vm 1
```

把 `VDOT_VV` 的表项插在 `VWADDU_VV` 前面，同一个机器码在硬件里就对应了两条语义不同的指令：译码按 `vdot` 走，而加宽指令的 `vd` 偶数对齐检查仍按 `vwaddu.vv` 生效。测试用了 `vd = 3`，直接判非法（`mcause = 2`）。

一个编码同时挂着两套语义，执行时必然出问题。

3.3 正确的验证方法：反汇编

把候选编码交给反汇编器，看有没有助记符：

```
0xC220A1D7  →  vwaddu.vv v3,v2,v1      有名字，说明已被占用
0xFA20A1D7  →  （无输出）               没有名字，才是空槽
```

反汇编器带的是完整的 RVV 指令表，比人工翻表可靠，也不需要知道表项叫什么名字。一条命令就能判定。

3.4 三条约束

反汇编之外，还要同时满足：

1. 香山译码表里没有对应表项；
2. 不与 RVV 1.0 已定义的指令冲突（含 rocket-chip 模板里的旧版编码）；
3. NEMU 的 `vopmvv` 表里同样为空。

第 3 条容易漏。NEMU 是参考模型，它按标准译码：如果它把这条编码识别成 `vwaddu.vv`，difftest 每一拍都在比较两条不同指令的结果，报错会指向一个并不存在的问题。

3.5 为什么是 `111110`

把 64 个 `funct6` 逐个过三条约束，在剩下的候选里取编号最大的一个，得到 `111110`（十进制 62）。

"取最大"是本项目为使选择过程可复现而定的规则：候选不止一个时，其他人按同样规则也能算出同一个答案。编号大小本身没有技术含义。

3.6 两条说明

先把三组编码字段摆在一起，都是 `opcode = 1010111`：

指令	funct3	funct6	与我们的关系
vdot.vv（本作品）	010	111110	—
vwmaccus.vx（RVV 1.0 已定义）	110	111110	funct6 相同，funct3 不同
Zvdot4a8i（未批准）	010	101000 / 101010 / 101100	funct3 相同，funct6 不同

第一条：vwmaccus.vx 与我们的 funct6 相同，但指令不一样。

funct3 是 110 对 010，差 3 位，拼出来的 32 位编码就不同。译码器按整个 32 位字匹配，两条指令各走各的，不会混。所以这一条**不是冲突**，列出来是因为查 RVV 编码表时看到 111110 已被占用，容易误判成撞车。

第二条：Zvdot4a8i 与我们的 funct6 不同，但它说明这段空间正在被动用。

它占的是 101000、101010、101100 三个槽，与 111110 不重叠，现在没有冲突。需要注意的"将来"：这是**未批准**的草案，如果它被批准并扩大范围，或者又有新扩展落到 111110，本编码就得让位。

4. 微架构

```
vs1(128)、vs2(128) 各取 8 个 int8
        │
        ├─ ① 输入符号扩展 8 → 16 位 (Chisel 的 SInt(8) * SInt(8) 行为)
        ▼
   16×16 有符号乘法器 × 8  ──►  8 个 16 位乘积
        │
        ▼
   ② 乘积符号扩展 16 → 32 位 (SInt.pad)
        │
        ▼
   8 个 32 位项 ──► 三级平衡树 8 → 4 → 2 → 1
        │
        ▼
   {96'b0, sum} ──► res.data
```

- 单元内没有寄存器，是纯组合逻辑；
- 整条指令按 `VimacCfg.latency = 2` 的流水 FU 执行，波形实测 t=8318 进、8320 出；
- 乘法器在 RTL 里是 **16×16**，不是 8×8。源码写的是 `(a.asSInt * b.asSInt).pad(32)`，其中 a、b 是 8 位；Chisel 会先把两个操作数符号扩展到 16 位再相乘，firtool 便按 16×16 生成。逻辑上只有低 8 位有效，高 8 位是符号复制。
- 乘积本身是 16 位。int8 × int8 的范围是 `[-16256, 16384]`，16 位有符号刚好装下。每个乘积在进入加法树之前先 `pad(32)`，所以树上处理的是 **8 个 32 位项**。

- 累加前必须符号扩展到 32 位。8 项求和最大 `131072`（18 位），16 位加法器在第 3、4 项就会溢出；同时负权重只有做符号扩展才能保持正确。符号扩展是连线，不增加乘法器成本，变宽的是加法树。
- 每级截断到 32 位不会丢掉有效位。int8 乘积的绝对值最大是 `(-128) × (-128) = 16384`，8 项求和最大 `131072`，即 2^{17}，只占 18 位（含符号位），离 32 位上限还差 14 位。**合法输入下这条指令不会溢出。**
- 仍然按"模 2^{32} 加法"来写，是因为语义定义成 32 位累加；这同时保证将来若把元素数放大，逐级截断与最后一次性截断的结果一致。

为什么不用现成的 `VIMac64b`：`VIMacU` 里有 2 个 `VIMac64b`（`numVecModule = dataWidth / 64`），每个处理 64 位、内含 8 个字节乘法器。它的宽化模式靠一个 `highHalf` 输入选择输出低半区还是高半区：一条 uop 只能给出 4 个 16 位乘积（64 位输出装不下 8 个），另外 4 个要由 `uopIdx+1` 那一拍补上。`vdot` 要一次凑齐 8 个，就得跨两个 uop 收集，再把高低半区拼回 16 位、符号扩展到 32 位，这段拼装需要改 `VIMacU` 的接线。

而这条接线同时供 `vmacc`、`vwmul` 使用，改动可能破坏旧指令，在后续考虑对这部分进行优化实现。

---

5. RTL 改动

让香山认识一条新指令，要接三处：

```
① 定义编码   →   ② 接进译码表   →   ③ 接进执行单元
Instructions.scala     VecDecoder.scala     VIMacU.scala
                       DecodeUnit.scala     VDotUnit.scala (新建)
```

另加一个全局开关 `hasVdot`，用于在"带 vdot"与"不带 vdot"两个构建之间切换，阶段三做性能对照要用。

5.1 定义编码（`decode/Instructions.scala`）

```
object VDot {
  def VDOT_VV = BitPat("b111110??????????010?????1010111")
}
```

位串的字段划分见 §3.1。这一处只提供编码，不涉及任何执行逻辑。

5.2 接进译码表（`VecDecoder.scala`、`DecodeUnit.scala`）

表项加一行，把编码和执行单元对应起来：

```
VDOT_VV -> OPMVV(F, FuType.vimac, VdotOpcode.vdot, F, T, F, VEC_VVV)
```

七个参数：

位置	值	含义
1	`F`	`vdRen`：不读旧 `vd`
2	`FuType.vimac`	交给乘加单元执行
3	`VdotOpcode.vdot`	单元内部的操作码
4–6	`F`、`T`、`F`	写标量 / 写向量 / 写饱和标志
7	`VEC_VVV`	uop 不拆分

开关在这里生效。译码表是 Scala 的 `List`，构造时把这一行滤掉即可：

```
decodeTable.filterNot(isVdotEntry)
```

5.3 接进执行单元（`VIMacU.scala`、新增 `VDotUnit.scala`）

`VIMacU` 原本的输出全部来自 `VIMac64b`，现在按 `hasVdot` 二选一：

```
if (hasVdot) {
  val vdot = Module(new VDotUnit)
  res.data     := Mux(isVdot, vdot.io.out, macData)
  res.vxsat    := Mux(isVdot, 0.U,         macVxsat)
  exceptionVec := Mux(isVdot, vdotExc,     macExc)
}
```

三处（结果、饱和标志、异常）都要选，漏一处就会混进 `VIMac64b` 的旧值。

`VDotUnit.scala` 是新增文件，107 行，乘法阵列 + 归约树，结构见 §4。

改动量：`git diff --numstat` 为 5 个已有文件 **+70 −5**，另加 1 个新文件。

5.4 三个联动点

其一，`vdRen = F` 与绕过 `Mgu` 是一对。 `Mgu` 的职责是"新算出的值与旧值合并"。译码已经声明不读旧 `vd`，就没有旧值可合并，输出必须整寄存器给值（高 96 位清零）。两处必须同时成立。

其二，`hasVdot` 要在三处同时生效。 `Parameters` 里定义、`DecodeUnit` 里过滤表项、`VIMacU` 里跳过例化。少改一处会留下半成品：译码表还在而单元没例化，表现为"能译码、执行时没有单元"；单元在而表项被滤掉，表现为"有单元、没有指令能进来"。

其三，`vm=0` 在编码层就被排除。 `BitPat` 把 bit25 固定写成 `1`，`vm=0` 的编码在译码阶段根本匹配不上，不会走到执行单元。（NEMU 侧的做法见阶段三文档。）

6. 测试

层	用例	期望
定向	正数递增 `1..8 × 1..8`	`0x000000CC`（204）
定向	全负数 `-1..-8 × 1..8`	`0xFFFFFF34`（−204）
定向	边界最小 `[-128]×8 × [-128]×8`	`0x00020000`（131072）

层	用例	期望
定向	边界最大 `[127]×8 × [127]×8`	`0x0001F808`（129032）
定向	`vl=8` / `vl=4` / `vl=0`	均为 `204`（`vl` 不参与）
非法	`vm=0` / `vsew=e16` / `vlmul=m2`	`mcause=2`
随机	100 组随机 int8	三方一致

期望值由独立的 C 实现产生，不是手算。非法用例只在 RTL 侧观察 `mcause`，因为 NEMU 的 trap 不经过 difftest 比对。

---

7. 与阶段一的 `vadd.vv` 对照

维度	`vadd.vv`	`vdot.vv`
机器码	`0x021101D7`	`0xFA20A1D7`
子类	OPIVV（`funct3=000`）	OPMVV（`funct3=010`）
功能单元	`VIAluFix`	`VIMacU`
FU 内拍数	1 拍	2 拍
数据操作	逐 lane 相加	8 个 lane 相乘后归约成 1 个标量
第三个源（旧 `vd`）	读	不读
过 `Mgu`	过	不过
用 `vl`	用	不用

阶段一测到的 `vadd` 是 1 拍，属于 `VIAluFix` 通路；`vdot` 的 2 拍属于 `VIMacU` 通路。两者的拍数不能互相套用。

8. 复现

```
bash scripts/repro-stage2.sh
```

脚本做四件事，任一失败退出码非 0，日志写进 `evidence/stage2/`：

1. **译码表互斥**：`scripts/check-decode-disjoint.py` 把表里 467 条 BitPat 两两比一遍，确认新加的 `VDOT_VV` 与其余 466 条没有重叠——这是"加一条指令不改变既有指令行为"的前提；
2. **反汇编核对**：把 `0xFA20A1D7`（本作品）、`0xFA20E1D7`（`funct3=110` 的 `vwmaccus.vx` 槽）、`0xC220A1D7`（`vwaddu.vv`）三条编码放进同一个 `.S`，看反汇编器把哪一条认成标准指令；
3. **定向功能**：`tests/vdot/vdot_basic_tests.c`，9 个用例，期望值由独立的 C 实现产生；
4. **非法与 `vstart`**：`vdot_illegal_tests.c`（`vm=0` / `e16` / `m2` 三条）、`vdot_vstart_test.c`（3 条），在 RTL 侧看 `mcause=2`。

RTL 侧跑批一律用 `timeout` 包住：被测程序结尾是 `wfi` 死循环，仿真器不会自己退出。